ESPECIFICAÇÃO TÉCNICA — SAGA ORQUESTRADA

Estudo comparativo de Sagas em microsserviços | Documento de implementação | 29 de outubro de 2026

Status: decisões de projeto adotadas para a primeira implementação. Não constitui resultado experimental nem substitui o plano de trabalho da IC.

1. Escopo, fontes e delimitação

Esta especificação orienta exclusivamente a construção da modalidade orquestrada. O plano de trabalho da IC (§3.1–3.2) estabelece o mesmo caso de liquidação de ordens para ambas as modalidades, com quatro serviços de domínio (ordens, conta, custódia e liquidação), RabbitMQ, Java/Spring Boot e PostgreSQL independente por serviço. A modalidade orquestrada inclui um coordenador técnico adicional. As regras de negócio aqui definidas deverão ser preservadas quando a modalidade coreografada for implementada; apenas o mecanismo de coordenação deverá mudar.

O protocolo-base (§3.3–3.5) confirma estados de domínio, invariantes e a ausência de isolamento global entre Sagas. Os nomes finais de eventos, contratos, esquemas de banco, regras financeiras simplificadas, confiabilidade e parâmetros de execução eram propostas em aberto. As definições correspondentes neste documento são decisões de engenharia adotadas para tornar o protótipo executável, e não afirmações de que já constavam do plano.

Objeto: compra simulada de um único ativo, por um cliente comprador, a partir de estoque de uma contraparte cadastrada. Não há negociação de preços em tempo real, integração com bolsa, conta bancária externa, transferências reais nem 2PC. A prioridade é comparar recuperação, compensações e consistência das Sagas em condições reprodutíveis.

2. Decisões de domínio que encerram as ambiguidades

Questão	Definição adotada	Justificativa
Entrada	POST /api/ordens com clienteId, ativoId e quantidade inteira positiva	Identifica o ativo; o custo é calculado pelo sistema.
Preço	Catálogo fixo e versionado, consultado no registro da ordem	Evita dependência de mercado externo e variação durante a Saga.
Contraparte	Estoque simulado, previamente abastecido, pertencente a uma contraparte fixa	Viabiliza reserva de ativo sem criar serviço de negociação.
Saldos	Dados iniciais configuráveis por cenário; sem endpoint depositar() no MVP	Permite insuficiência de saldo e concorrência controladas.
Liquidação	Marco durável de confirmação; depois dele, concluir efetivações pendentes	Evita compensar uma operação declarada liquidada.
Outbox / DLQ	Outbox transacional, consumidor idempotente, filas	Cobertura para falhas entre commit e publicação ou

	de erro e recuperação persistente	consumo.
Parâmetros	Timeouts, retries e carga configuráveis; valores medidos no piloto	Não antecipar números experimentais sem evidência.

Os valores de catálogo, saldo e estoque são dados de teste; sua preparação e seus arquivos de seed devem ser versionados. Uma alteração de preço posterior à recepção da ordem não altera o preço capturado na ordem. Valores monetários usam NUMERIC e BigDecimal (não float/double).

3. Responsabilidades e limites de dados

Componente	**Responsabilidade exclusiva**	**Armazenamento local**
ordens-service	Entrada HTTP; validação da solicitação; consulta de catálogo; fixação do preço; status da ordem; consulta do resultado	ordens, catalogo_ativos, idempotencia_http, outbox, inbox
conta-service	Disponibilidade de saldo do comprador; reserva; liberação; consumo definitivo	contas, reservas_saldo, outbox, inbox
custodia-service	Posições de estoque da contraparte; reserva; liberação; transferência simulada ao comprador	posicoes, reservas_ativo, outbox, inbox
liquidacao-service	Gravação idempotente do compromisso durável de liquidação, após as duas reservas	liquidacoes, outbox, inbox
orquestrador-service	Sequenciamento; correlação; estado técnico; timeouts, repetição, reconciliação e compensações	sagas, etapas_saga, outbox, inbox

Cada serviço possui banco PostgreSQL próprio (instância ou isolamento operacional equivalente com credenciais e esquemas exclusivos); nenhum serviço consulta diretamente as tabelas de outro. O orquestrador não calcula saldo, preço ou estoque. A consulta inicial ao catálogo é feita localmente por ordens; a versão do catálogo e o preço capturado acompanham a Saga.

4. Identificadores, semântica da API e idempotência

Campo	**Regra**
orderId	UUID de negócio gerado uma única vez pelo serviço de ordens; UNIQUE.
sagaId	UUID gerado para a execução distribuída;

	referência em todas as mensagens.
operationId	UUID determinístico por sagaId + tipo de etapa, ou valor estável persistido ao criar cada etapa; não muda em retries.
messageId	UUID por envelope/publicação; pode mudar em reenvios, sem mudar operationId.
schemaVersion	Inteiro inicial 1, validado por consumidores.
Idempotency-Key (HTTP)	Obrigatória em POST /api/ordens; unicidade por cliente + chave; payload divergente na mesma chave recebe 409.
correlationId / causationId	Não criar como campos obrigatórios no MVP; sagaId correlaciona o fluxo e tracing cobre a causalidade.

POST /api/ordens recebe {clienteId, ativoId, quantidade}. Responde 202 Accepted com orderId, sagaId, estado RECEBIDA e URI de consulta. GET /api/ordens/{orderId} devolve estado e desfecho; GET /api/sagas/{sagaId} deve ficar restrito a diagnóstico, sem divulgar dados internos. Retentativa HTTP com a mesma chave e mesmo payload retorna o registro original, sem criar segunda Saga. Identificadores de cliente e ativo podem ser strings estáveis dos dados de teste; não precisam ser UUIDs.

```
POST /api/ordens
Idempotency-Key: <chave>
{"clienteId":"CLI001","ativoId":"ATV002","quantidade":3}

202 {"orderId":"<uuid>","sagaId":"<uuid>","estado":"RECEBIDA"}
```

5. Modelo financeiro simplificado e invariantes

Na conta do comprador, saldoDisponivel e saldoReservado são grandezas distintas. Reservar V exige saldoDisponivel >= V e executa, atomicamente: disponivel -= V; reservado += V. Compensar libera exatamente a reserva identificada: disponivel += V; reservado -= V. Confirmar consome a reserva: reservado -= V; totalConsumido += V, sem segunda redução do saldo disponível.

Na custódia, quantidadeDisponivel e quantidadeReservada do estoque da contraparte seguem a mesma lógica. Confirmar reduz a quantidade reservada da contraparte e aumenta a posição efetivada do comprador na mesma transação local. Nenhuma confirmação pode repetir a transferência. A operação ConfirmarReservaAtivo só é aceita se existir reserva previamente criada para a ordem.

Invariantes do protocolo-base (§3.4): nenhuma ordem LIQUIDADA sem efetivação de conta e custódia; nenhuma reserva ativa remanescente após REJEITADA; reserva de ativo dependente de reserva de saldo compatível; disponibilidade de saldo e estoque nunca negativa; unicidade de orderId e compensação da reserva original. A verificação final ocorre após a janela de convergência, pois estados transitórios podem diferir durante a execução.

Estados de domínio definidos no protocolo-base (§3.3): ordens RECEBIDA → LIQUIDADA ou REJEITADA; conta SALDO_RESERVADO → CONFIRMADO ou RESERVA_DESFEITA; custódia ATIVO_RESERVADO → CONFIRMADO ou RESERVA_DESFEITA; liquidação CONFIRMADA (ou

registro ausente). Não acrescentar estados de domínio sem necessidade; usar estados técnicos separados para passos em trânsito.

6. Máquina de estados do orquestrador

Estado técnico	Condição / comportamento
CRIADA	Ordem recebida e Saga registrada, ainda sem comando de saldo concluído.
RESERVANDO_SALDO	Comando emitido; resposta pendente ou desconhecida.
RESERVANDO_ATIVO	Saldo confirmado como reservado; reserva do ativo pendente.
CONFIRMANDO_LIQUIDACAO	Ambas reservas confirmadas; liquidação ainda não confirmou duravelmente.
EFETIVANDO	Liquidação confirmada; efetivações de conta e custódia aguardadas.
COMPENSANDO	Falha definitiva pré-liquidação; desfazer reservas em ordem inversa.
CONCLUIDA	Liquidação durável e duas efetivações confirmadas; ordem LIQUIDADA.
REJEITADA	Falha pré-liquidação e todas as compensações necessárias confirmadas; ordem REJEITADA.
RECUPERACAO_PENDENTE	Resultado desconhecido, retry esgotado ou operação pendente; não declarar terminal.

A liquidação é a barreira de decisão. Antes de CONFIRMADA, o orquestrador pode compensar. Após o registro durável de CONFIRMADA, a recuperação obrigatoriamente segue para efetivar reservas, nunca para liberá-las automaticamente. Uma falha de comunicação após o commit é tratada como resultado desconhecido e resolvida por consulta idempotente, não por suposição.

7. Fluxo operacional completo (caminho feliz)

1. Ordens recebe POST, valida entrada, resolve preço, gera orderId/sagaId e persiste ordem RECEBIDA e evento interno de início no mesmo commit local.
2. O orquestrador consome OrdemRecebida, cria execução idempotentemente e publica ReservarSaldo com operationId estável.
3. Conta valida saldo, cria reserva, atualiza saldos e publica SaldoReservado; recusas publicam SaldoRecusado com motivo.
4. O orquestrador recebe SaldoReservado e publica ReservarAtivo.
5. Custódia verifica estoque, cria reserva e publica AtivoReservado ou AtivoRecusado.
6. O orquestrador publica ConfirmarLiquidacao; liquidação grava registro CONFIRMADA de forma idempotente e publica LiquidacaoConcluida.

7. O orquestrador passa a EFETIVANDO e publica ConfirmarReservaSaldo e ConfirmarReservaAtivo (podem ocorrer independentemente após a decisão durável).
8. Conta e custódia efetivam suas reservas exatamente uma vez em termos de efeito, retornam SaldoConfirmado e AtivoConfirmado.
9. O orquestrador, após ambos os resultados, publica FinalizarOrdem com resultado LIQUIDADA. Ordens grava LIQUIDADA e responde OrdemLiquidada.
10. O orquestrador registra CONCLUIDA somente após OrdemLiquidada; GET da ordem permite consultar o resultado final.

8. Contratos de mensagens — versão 1

Publicação assíncrona em RabbitMQ por exchanges de comandos e respostas. Todas as mensagens incluem type, schemaVersion, messageId, sagaId, orderId, operationId, occurredAt e payload. Comandos são roteados a um serviço específico. Respostas são roteadas ao orquestrador; tipos de mensagem são versionados. Cada consumidor rejeita de forma recuperável versões incompatíveis e registra erros de contrato em DLQ, sem executar efeitos de domínio.

Comando / origem	**Resposta / origem**	**Efeito**
OrdemRecebida / ordens	—	Iniciar Saga idempotentemente.
ReservarSaldo / orquestrador	SaldoReservado | SaldoRecusado / conta	Reserva monetária.
ReservarAtivo / orquestrador	AtivoReservado | AtivoRecusado / custódia	Reserva de estoque.
ConfirmarLiquidacao / orquestrador	LiquidacaoConcluida | LiquidacaoFalhou / liquidação	Marco durável da liquidação.
ConfirmarReservaSaldo / orquestrador	SaldoConfirmado / conta	Consumo da reserva monetária.
ConfirmarReservaAtivo / orquestrador	AtivoConfirmado / custódia	Transferência simulada do ativo.
DesfazerReservaAtivo / orquestrador	ReservaAtivoDesfeita / custódia	Liberar somente a reserva original.
DesfazerReservaSaldo / orquestrador	ReservaSaldoDesfeita / conta	Liberar somente a reserva original.
FinalizarOrdem / orquestrador	OrdemLiquidada | OrdemRejeitada / ordens	Persistir desfecho final.

Erros temporários de infraestrutura não são respostas de recusa de negócio: devem gerar retry e recuperação. LiquidacaoFalhou só significa ausência comprovada de decisão durável; caso o resultado seja desconhecido, consultar pelo orderId/sagaId antes de compensar. Em todos os casos, payload deve carregar identificadores do recurso, valor, quantidade, preço capturado e código de motivo quando aplicável. Manter um catálogo de códigos de erro estáveis, por exemplo SALDO_INSUFICIENTE, ATIVO_INSUFICIENTE e ENTRADA_INVALIDA.

9. Falhas, compensações e recuperação

Ponto de falha	Ação correta
Validação / registro de ordem	Rejeitar a requisição sem iniciar Saga; sem compensação.
Reserva de saldo recusada	Marcar REJEITADA, sem compensar recursos inexistentes.
Reserva de ativo recusada	Confirmar ausência de reserva parcial em custódia; desfazer reserva de saldo; então REJEITADA.
Liquidação negada com ausência comprovada de commit	Desfazer ativo e depois saldo; então REJEITADA.
Resposta de reserva ou liquidação perdida	Consultar resultado pelo operationId e reenviar idempotentemente, sem decisão por timeout isolado.
Falha durante compensação	Registrar COMPENSANDO/RECUPERACAO_PENDENTE; reexecutar até confirmar ausência de reserva ativa.
Falha após liquidação CONFIRMADA	Nunca compensar automaticamente; recuperar saldo/ativo e finalizar ordem.
Duplicação de mensagens	Responder com resultado previamente persistido; sem repetir efeito.
Crash entre commit e publish	Outbox registra publicação pendente; publicador reenvia até confirmação.

Recuperação após reinício: varredura periódica de Sagas não terminais e operações com prazo expirado. A rotina consulta os estados persistentes, decide entre retomar, consultar resultado ou compensar e publica somente operações idempotentes. O resultado de cada compensação também é persistido. Não há encerramento terminal de Saga com compensação ou efetivação pendente.

10. Persistência, isolamento e concorrência

Banco	Tabelas mínimas
Ordens	orders; asset_catalog; http_idempotency; outbox_messages; inbox_operations
Conta	accounts; balance_reservations; outbox_messages; inbox_operations
Custódia	positions; asset_reservations; outbox_messages; inbox_operations
Liquidação	settlements; outbox_messages; inbox_operations

Orquestrador	saga_instances; saga_steps; outbox_messages; inbox_operations

Cada reserva armazena orderId, sagaId, operationId, recurso, quantidade ou valor, status e timestamps, com UNIQUE para impedir dupla reserva por Saga/etapa. A liquidação usa UNIQUE(orderId) para impedir dupla decisão. Em PostgreSQL, iniciar com READ COMMITTED e UPDATE condicional (saldo_disponivel >= valor; estoque_disponivel >= quantidade), na mesma transação que insere a reserva. Utilizar lock/controle otimista apenas onde necessário; evitar leitura seguida de escrita sem condição atômica. Deadlocks são falhas transitórias recuperáveis.

Outbox transacional em cada serviço: atualização local e registro de mensagem no mesmo commit. Publicador periódico envia mensagens persistentes com publisher confirms e marca entrega depois do ACK do broker. A entrega é pelo menos uma vez; inbox/idempotência garante efeito único da operação. ACK do consumidor só ocorre após commit local. Falhas irrecuperáveis de formato seguem para DLQ e monitoramento; operações financeiras pendentes permanecem registradas e reconciliáveis.

11. Infraestrutura, configuração e interface de desenvolvimento

Monorepositório com orquestrada/, coreografada/ (ainda não implementada), compartilhado/ (apenas contratos e regras puras), infra/, experimentos/ e docs/. A primeira entrega de código utilizará Maven multimódulo por variante; Java/Spring Boot em versões estáveis compatíveis fixadas no POM; RabbitMQ e PostgreSQL via Docker Compose; migrações de esquema versionadas (Flyway ou equivalente); nenhuma senha embutida no código. README deve conter inicialização, endpoints, diagramas e comandos de teste.

Properties configuráveis: saga.timeout.step; saga.retry.max-attempts; saga.retry.initial-delay; saga.retry.max-delay; outbox.poll-interval; reconciliation.poll-interval; rabbitmq.prefetch; e parâmetros de pool de conexão. Valores operacionais padrão serão usados apenas para desenvolvimento; valores experimentais exigem piloto. Incluir health/readiness, logs estruturados contendo sagaId/orderId/operationId, OpenTelemetry e métricas Prometheus.

12. Critérios de aceitação e testes antes da coreografia

- Contrato HTTP cria uma única ordem sob reenvio da mesma Idempotency-Key; chave igual e payload diferente recebe 409.
- Caminho feliz conclui uma ordem e confirma saldo/ativo uma única vez, com registro de liquidação e ausência de reservas pendentes.
- Saldo insuficiente, estoque insuficiente e falha pré-liquidação não deixam recursos reservados após compensação concluída.
- Concorrência de ordens sobre a mesma conta/posição não produz disponibilidade negativa nem dupla alocação.
- Injeções de falha C1–C6 do protocolo-base exercitam crash, timeout, recusa, duplicação, ausência de publicação e falha parcial.
- Crash entre commit e publicação é recuperado pela Outbox; reenvios não repetem movimentações.
- Crash após liquidação confirmada não reverte reservas; recuperação completa efetivações e marca ordem LIQUIDADA.
- Após reinício dos serviços, todas as Sagas incompletas são identificáveis e recuperáveis; estados não terminais não são ocultados.

- Sistema oferece consulta de estado e logs/telemetria para medir tempo de convergência, compensações e latência.
- Seed, configurações, cenários e versão de código são reprodutíveis; testes de carga só são parametrizados após piloto.

13. Sequência de execução no Work

11. Criar novo repositório e estrutura orquestrada/; configurar .gitignore Java/Maven, Docker Compose, PostgreSQL separado por serviço, RabbitMQ e migrações.
12. Fixar contratos de domínio, eventos e comandos; implementar entidades, constraints e testes locais das reservas.
13. Implementar API de ordens e catálogo fixo, saldo inicial e estoque configurável.
14. Implementar consumidores idempotentes, Outbox e publicação de comandos/respostas.
15. Implementar motor de Saga persistente e caminho feliz até confirmação e efetivação.
16. Implementar compensações em ordem inversa, recuperação pós-crash e reconciliador.
17. Executar testes integrados de falhas e concorrência; documentar métricas e preparar piloto.
18. Usar a versão orquestrada como referência funcional para implementar coreografada posteriormente.

14. Rastreabilidade das decisões

Origem documental	**Conteúdo aproveitado**	**Decisão adicional nesta especificação**
Plano da IC §§3.1–3.2	Quatro serviços, comparação de duas modalidades e stack	API concreta; catálogo e contraparte simulados.
Protocolo-base §§3.3–3.5	Estados e invariantes confirmados; concorrência e idempotência	Operações de reserva, efetivação e tabelas persistentes.
Protocolo-base §§4–6	Fluxos e falhas candidatos	Contratos v1; regra de commit durável da liquidação.
Protocolo-base §7	Retry, Outbox, DLQ ainda em aberto	Escolha por Outbox + inbox idempotente + recuperação persistente.
Plano §§3.3–3.10	Equivalência experimental, piloto, métricas e reprodutibilidade	Critérios de aceite e sequência de implementação.
PDF corrigido pelo orientador, pp. 3–5	Questionamentos sobre identificadores, API, saldos, liquidação, confiabilidade e parâmetros	Respostas operacionais detalhadas nas seções 2–12.

Limite de escopo: este documento fecha decisões de engenharia para a primeira implementação orquestrada. Não é um protocolo experimental aprovado nem apresenta medições. O estudo posterior deverá manter as mesmas semânticas e critérios na variante coreografada.